

MANUAL TÉCNICO: FreelanceHub

1. Introducción

El presente documento detalla la arquitectura, tecnologías y estructura interna de la plataforma **FreelanceHub**, un sistema web diseñado para conectar Clientes y Freelancers. El sistema opera bajo una arquitectura Cliente-Servidor separando el Frontend (Single Page Application) y el Backend (API RESTful).

2. Arquitectura del Sistema

El proyecto está construido bajo una arquitectura de N-Capas:

- **Capa de Presentación (Frontend):** Desarrollada en Angular. Maneja el ruteo del lado del cliente, la gestión de estados (Signals) y el consumo de la API.
- **Capa de Negocio (Backend):** Desarrollada en Java puro utilizando JAX-RS para la exposición de Endpoints REST.
- **Capa de Acceso a Datos (DAO):** Implementación de JDBC puro para la comunicación directa con el motor de base de datos.
- **Capa de Datos:** Base de datos relacional MySQL.

3. Stack Tecnológico

- **Frontend:** Angular (con la nueva sintaxis de control de flujo `@if`, `@for`), Bootstrap 5 (CSS/UI), HTML5.
- **Backend:** Java, JAX-RS (Jakarta EE), Servidor Apache Tomcat.
- **Base de Datos:** MySQL (Dialecto InnoDB para soporte de transacciones).
- **Entorno Recomendado:** SO Linux (Pop!_OS / Ubuntu) o Windows, IDE IntelliJ IDEA / VS Code.

4. Estructura de la Base de Datos (Modelo Relacional)

La base de datos Proyecto2 está normalizada y se compone de entidades clave:

- **Gestión de Usuarios:** Tabla usuario centralizada con herencia estructural hacia cliente y freelancer (Relación 1:1).
- **Gestión Financiera:** Tablas cartera, transaccion, cartera_plataforma y transaccion_plataforma para rastrear recargas, pagos, bloqueos de fondos y cálculo de comisiones del sistema.

- **Flujo de Negocio:**
 - proyecto: Publicaciones realizadas por los clientes.
 - propuesta: Ofertas económicas de los freelancers.
 - contrato: Generado automáticamente al aceptar una propuesta.
 - entrega: Archivos/links subidos por el freelancer para el cumplimiento del contrato.
- **Catálogos Dinámicos:** categoría, habilidad y la tabla temporal `nueva_h_c` para el flujo de moderación del administrador.

5. Estructura del Backend (Java API)

El código fuente está organizado en los siguientes paquetes principales:

- **models / dto:** Clases planas (POJOs) para el mapeo de la DB y transferencia de datos (Requests/Responses). Ej: `ContratoResponse`, `ReporteProyectoDTO`.
- **dao:** Clases Data Access Object con sentencias SQL preparadas (`PreparedStatement`) para evitar inyección SQL. Manejan el CRUD.
- **services:** Contienen la lógica de negocio. Ej: Bloquear el saldo de la cartera de un cliente cuando acepta una propuesta, o calcular la comisión del sistema al finalizar un contrato.
- **resources / controllers:** Endpoints REST anotados con `@Path`, `@GET`, `@POST`, `@PUT`. Protegidos mediante anotaciones personalizadas de seguridad (Ej. `@Secured`).

6. Estructura del Frontend (Angular)

- **services/:** Contienen llamadas `HttpClient` hacia el backend. Ej: `auth.service.ts`, `contrato.service.ts`.
- **models/:** Interfaces TypeScript que espejean los DTOs de Java para tener tipado estricto.
- **components/:** Componentes reutilizables de UI (Smart/Dumb components). Ej: `solicitud-card`, `navbar`.
- **pages/:** Vistas enrutables completas agrupadas por rol de usuario (`admin/`, `cliente/`, `freelancer/`).
- **Ruteo (`app.routes.ts`):** Protegido mediante Guards para evitar que un Freelancer acceda a rutas de Cliente y viceversa.